

NTV 2026

Text Generation

Lecturer: Wilker Aziz
module 4, lecture a



Where are we at?

- First half
 - Statistics for NLP
 - Text classification and regression
 - Feature learning
 - Midterm
- Second half
 - Text generation / language modelling

Can you think of applications that require computers to ***generate*** text?

chatbot

2 responses



summarization



claude



Computer vision



chatbots



Making summaries



chatgpt



Translators



Google translate



There's no correct answer!

Can you think of applications that require computers to ***generate*** text?

chatbot

2 responses



summarization



claude



Computer vision



chatbots



Making summaries



chatgpt



Translators



Google translate

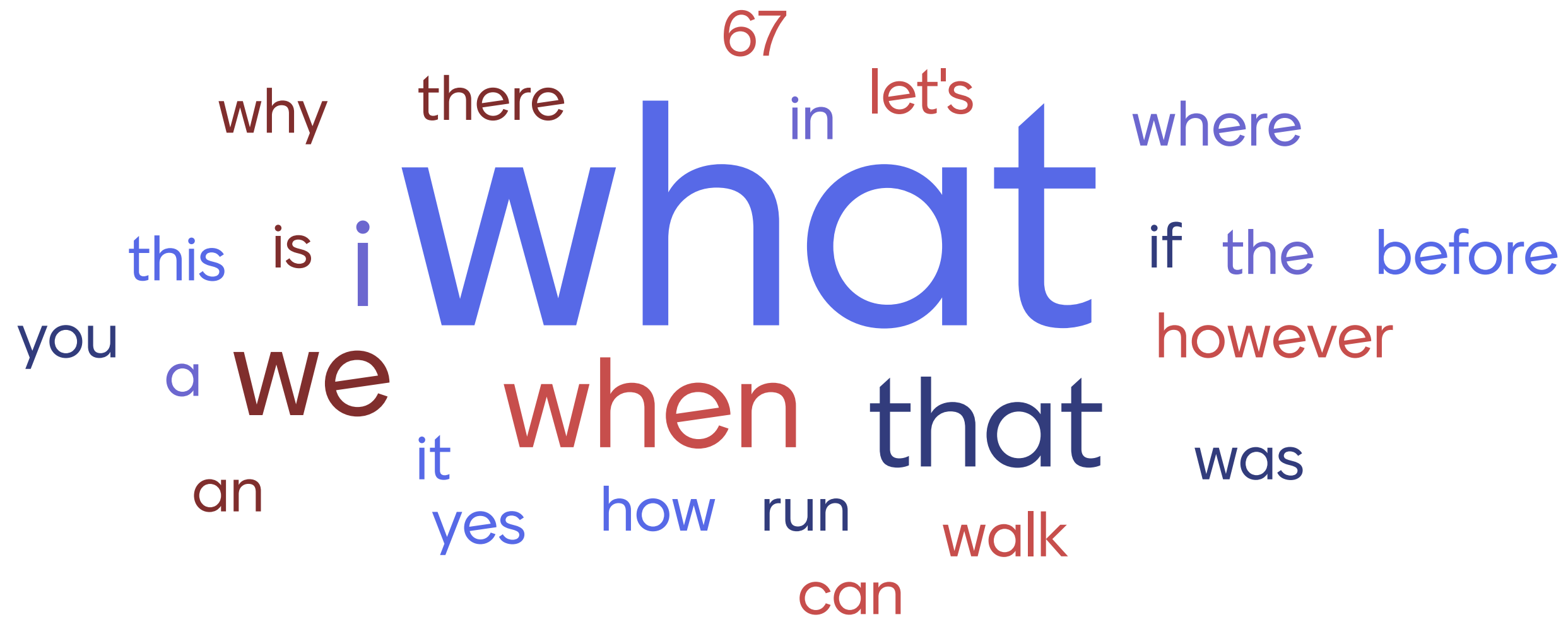


There's no correct answer!

Applications

- Autocompletion
- Autocorrection
- Translation
- Question answering
- Dialogue
- Essay generation
- ...

Start a sentence in English



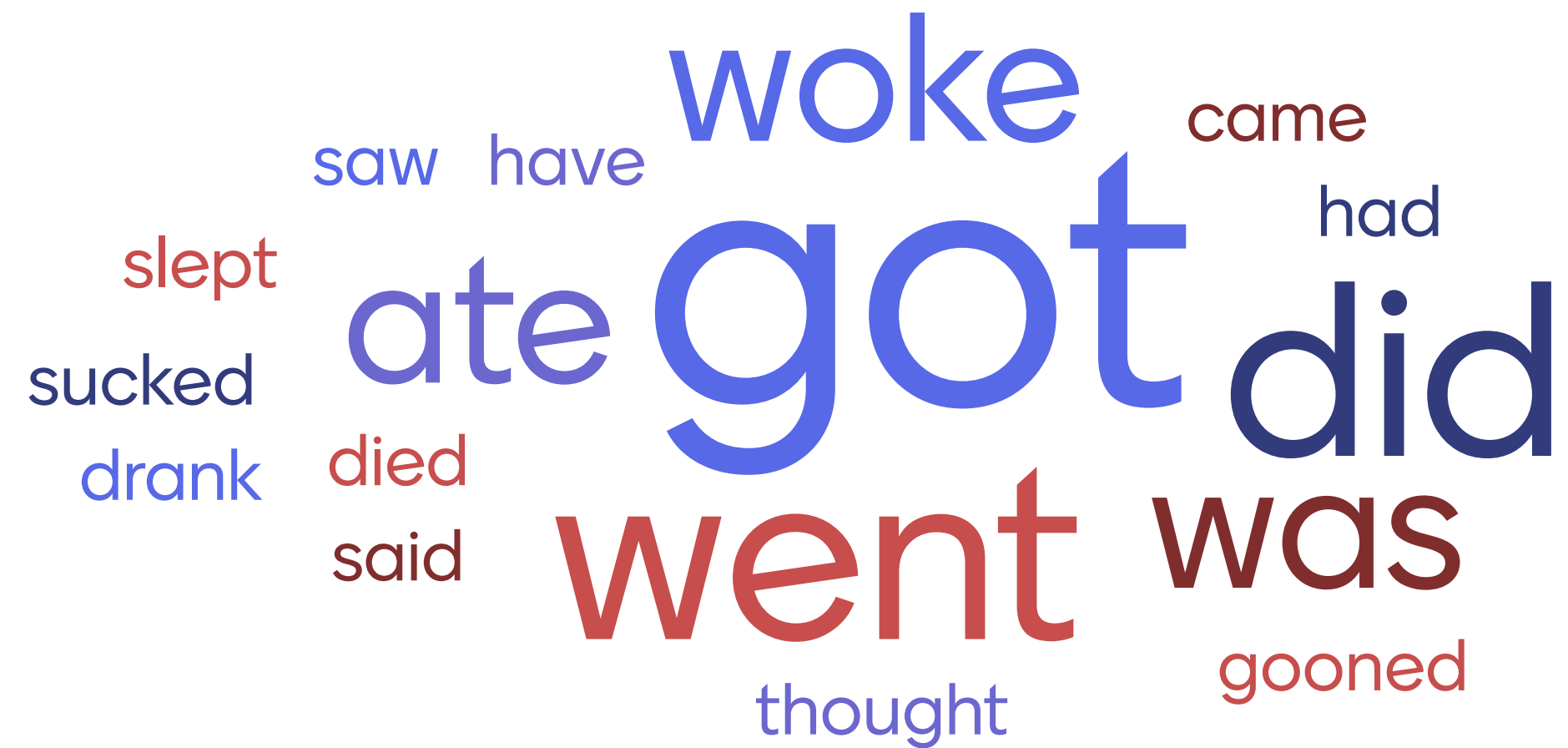
Continue this sentence: Yesterday ____

Word cloud visualization of responses to "Continue this sentence: Yesterday ____". The words are arranged in a circular pattern around a central point, with colors alternating between red and blue.

Words included in the word cloud:

- we
- the
- was
- there
- you
- is
- when
- they
- had
- evening
- died
- all
- seems
- did
- i
- in

Continue this sentence: Yesterday I ____



Continue this sentence: Yesterday I learnt ____

to

nlp

the

Continue this sentence: Yesterday I learnt about ____



Essential building block

We will build text generating applications from a mechanism capable of extending existing pieces of text one token at a time.

This mechanism is what we call a *language model*.

Outline

- Language model
- NGram LM
 - Classic
 - Neural
- Evaluation

Language Model

Probability distribution over texts.

For simplicity, a language is the set of all sequences made of symbols from a fixed finite vocabulary $\mathcal{W}$ (of size V).

With a large vocabulary, fluent texts are typically within the support of a language model.

Goal

Learn a probability distribution
whose samples *resemble* real,
observed text.

More technically, statistics of
generated data should distribute like
statistics of observed data.

Formalisation

W is a random *word*. An outcome w is a symbol in a vocabulary $\mathcal{W}$ of size V .

$X = \langle W_1, \dots, W_L \rangle$ is a random *sequence* of L words. We can also denote it $W_{1:L}$. An outcome $\langle w_1, \dots, w_l \rangle$ is a sequence of l symbols from $\mathcal{W}$.

The **challenge** is to express the probability

$$P(W_{1:L} = \langle w_1, \dots, w_l \rangle)$$

for **any** outcome $w_{1:l} \in \mathcal{W}^*$

Challenge

The space of all texts is infinite. A text is a discrete data point, but it's not an atomic category, it's a structured observation made of a sequence of tokens.

We do not know a distribution over infinite sequences, do we?

Generative story

1. Start with an empty history $H = \langle \rangle$. Set $i = 1$.
2. Condition on the available history $w_{<i}$ and draw a word w_i with probability $P(W = w_i | H = w_{<i})$ extending the history.
3. If w_i is EOS terminate, else increment i and repeat (2).

This specifies a **factorisation** of P_X in terms of factors of the kind $P_{W|H}$.

Formalisation

W is a random *word*. An outcome w is a symbol in a vocabulary $\mathcal{W}$ of size V .

$X = \langle W_1, \dots, W_L \rangle$ is a random *sequence* of L words. We can also denote it $W_{1:L}$. An outcome $\langle w_1, \dots, w_l \rangle$ is a sequence of l symbols from $\mathcal{W}$.

$W_{<i}$ is a random *history*. We sometimes denote it by H . An outcome $w_{<i} = \langle w_1, \dots, w_{i-1} \rangle$ is a sequence of $i - 1$ symbols from $\mathcal{W}$. When $i \leq 1$, $w_{<i}$ is the empty sequence.

Chain rule

We apply chain rule

$$\begin{aligned} P(X = w_{1:l}) \\ = \prod_{i=1}^l P(W = w_i | H = w_{<i}) \end{aligned}$$

which re-expresses the probability of a text using probabilities of words in context.

$$\begin{aligned} P_X(\langle \text{He, went, to, the, store, EOS} \rangle) &= P_{W|H}(\text{He}|\langle \rangle) \\ &\times P_{W|H}(\text{went}|\langle \text{He} \rangle) \\ &\times P_{W|H}(\text{to}|\langle \text{He, went} \rangle) \\ &\times P_{W|H}(\text{the}|\langle \text{He, went, to} \rangle) \\ &\times P_{W|H}(\text{store}|\langle \text{He, went, to, the} \rangle) \\ &\times P_{W|H}(\text{EOS}|\langle \text{He, went, to, the, store} \rangle) \end{aligned}$$

Example

$$\begin{aligned} P_X(\langle \text{He, went, to, the, store, EOS} \rangle) &= P_{W|H}(\text{He}|\langle \rangle) \\ &\times P_{W|H}(\text{went}|\langle \text{He} \rangle) \\ &\times P_{W|H}(\text{to}|\langle \text{He, went} \rangle) \\ &\times P_{W|H}(\text{the}|\langle \text{He, went, to} \rangle) \\ &\times P_{W|H}(\text{store}|\langle \text{He, went, to, the} \rangle) \\ &\times P_{W|H}(\text{EOS}|\langle \text{He, went, to, the, store} \rangle) \end{aligned}$$

Do you see any problem with this idea?

Summary

- P_X : we do not know a classic distribution over sequences
- $P_{W|H=h}$: we know a classic distribution over V categories
- But there's no limit to the set of possible histories

Outline

- Language model
- **NGram LM**
 - Classic
 - Neural
- Evaluation

How can we deal with or circumvent the need to deal with a growing history?



$$P_X(\langle \text{He, went, to, the, store, EOS} \rangle) \stackrel{\text{ind.}}{=} P_W(\text{He}) \\ \times P_W(\text{went}) \\ \times P_W(\text{to}) \\ \times P_W(\text{the}) \\ \times P_W(\text{store}) \\ \times P_W(\text{EOS})$$

Unigram LM

We could say words are independent of one another.

Though, is it realistic?

$$\begin{aligned} P_X(\langle \text{He, went, to, the, store, EOS} \rangle) &\stackrel{\text{ind.}}{=} P_{W|H}(\text{He}|\langle \rangle) \\ &\times P_{W|H}(\text{went}|\langle \text{He} \rangle) \\ &\times P_{W|H}(\text{to}|\langle \text{went} \rangle) \\ &\times P_{W|H}(\text{the}|\langle \text{to} \rangle) \\ &\times P_{W|H}(\text{store}|\langle \text{the} \rangle) \\ &\times P_{W|H}(\text{EOS}|\langle \text{store} \rangle) \end{aligned}$$

Bigram LM

Maybe we say a word is independent of every other word except the last?

$$\begin{aligned} P_X(\langle \text{He, went, to, the, store, EOS} \rangle) &\stackrel{\text{ind.}}{=} P_{W|H}(\text{He}|\langle \rangle) \\ &\times P_{W|H}(\text{went}|\langle \text{He} \rangle) \\ &\times P_{W|H}(\text{to}|\langle \text{He went} \rangle) \\ &\times P_{W|H}(\text{the}|\langle \text{went to} \rangle) \\ &\times P_{W|H}(\text{store}|\langle \text{to the} \rangle) \\ &\times P_{W|H}(\text{EOS}|\langle \text{the store} \rangle) \end{aligned}$$

Trigram LM

Maybe it should depend on the
previous 2 words?

Markov assumption

The key idea in the NGram LM is to make a conditional independence assumption:

$$P(X = w_{1:l}) \\ = \prod_{i=1}^l P(W = w_i | H = \textcolor{green}{w_{i-n+1:i-1}})$$

shortened past

a word is independent on all but the recent history of $n-1$ words. This is the so-called **Markov assumption of order $n-1$** .

Outline

- Language model
- NGram LM
 - Classic
 - Neural
- Evaluation

Factorisation

$$P(X = w_{1:l}) \stackrel{\text{ind.}}{=} \prod_{i=1}^l P(W = w_i | H = w_{i-n+1:i-1}) .$$

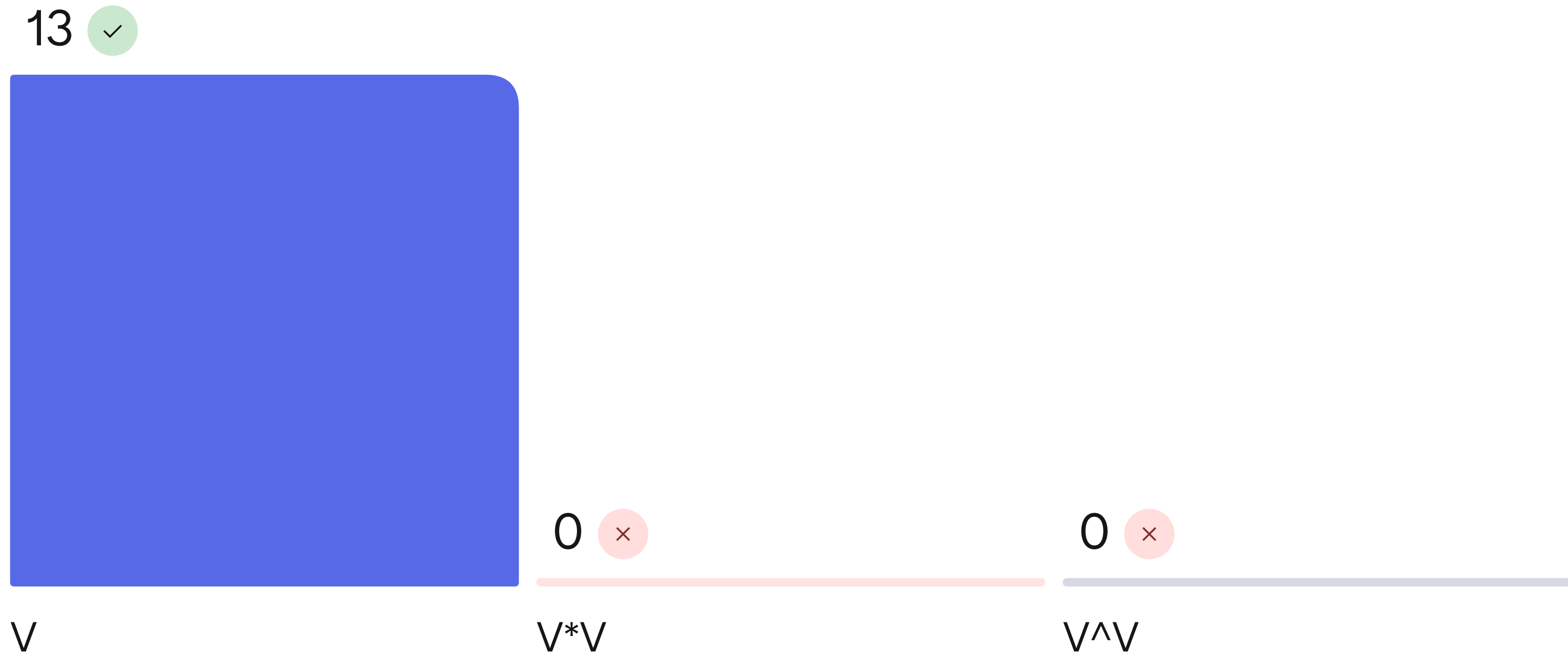
Parameterisation: for each unique history $h \in \mathcal{W}^{n-1}$

$$W | H = h \sim \text{Categorical}(\pi_{1:V}^{(h)}) .$$

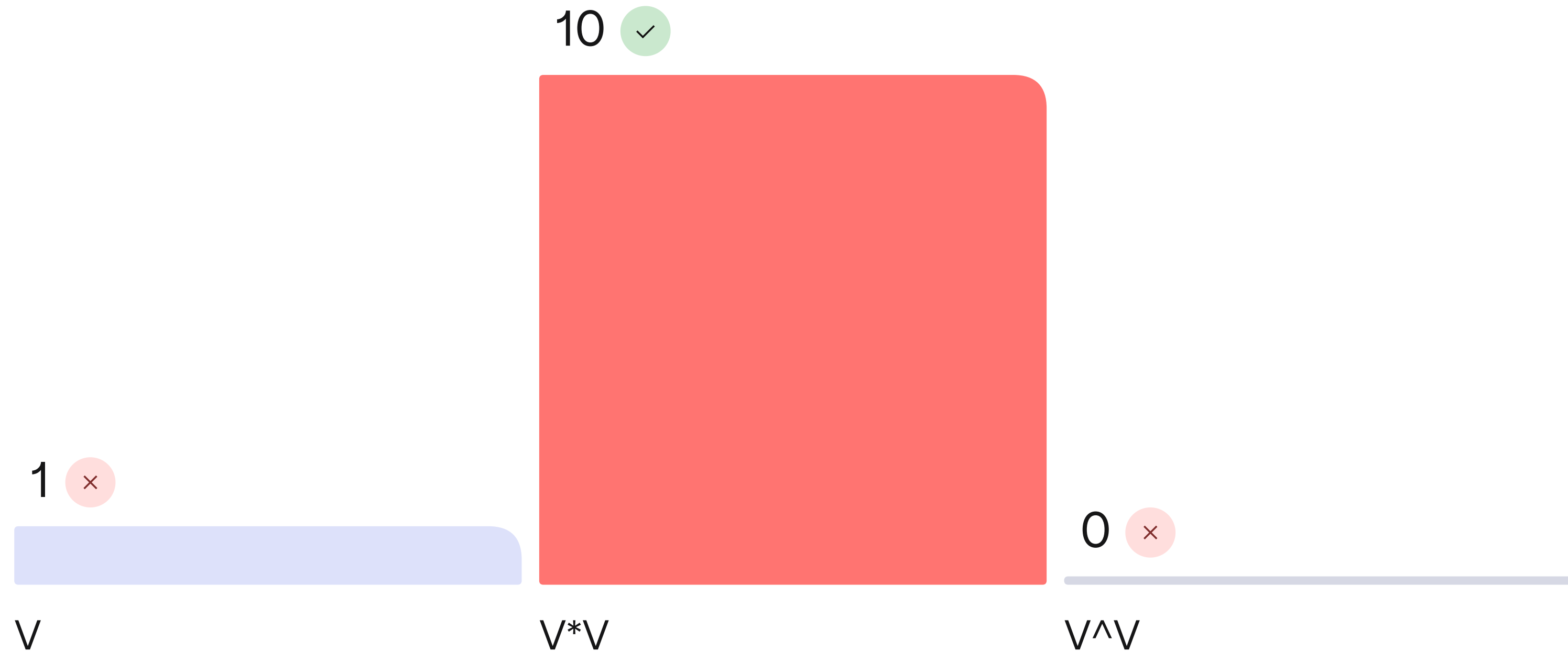
Tabular cpds

Employ a tabular representation of the conditional probability distributions (cpds) in the model

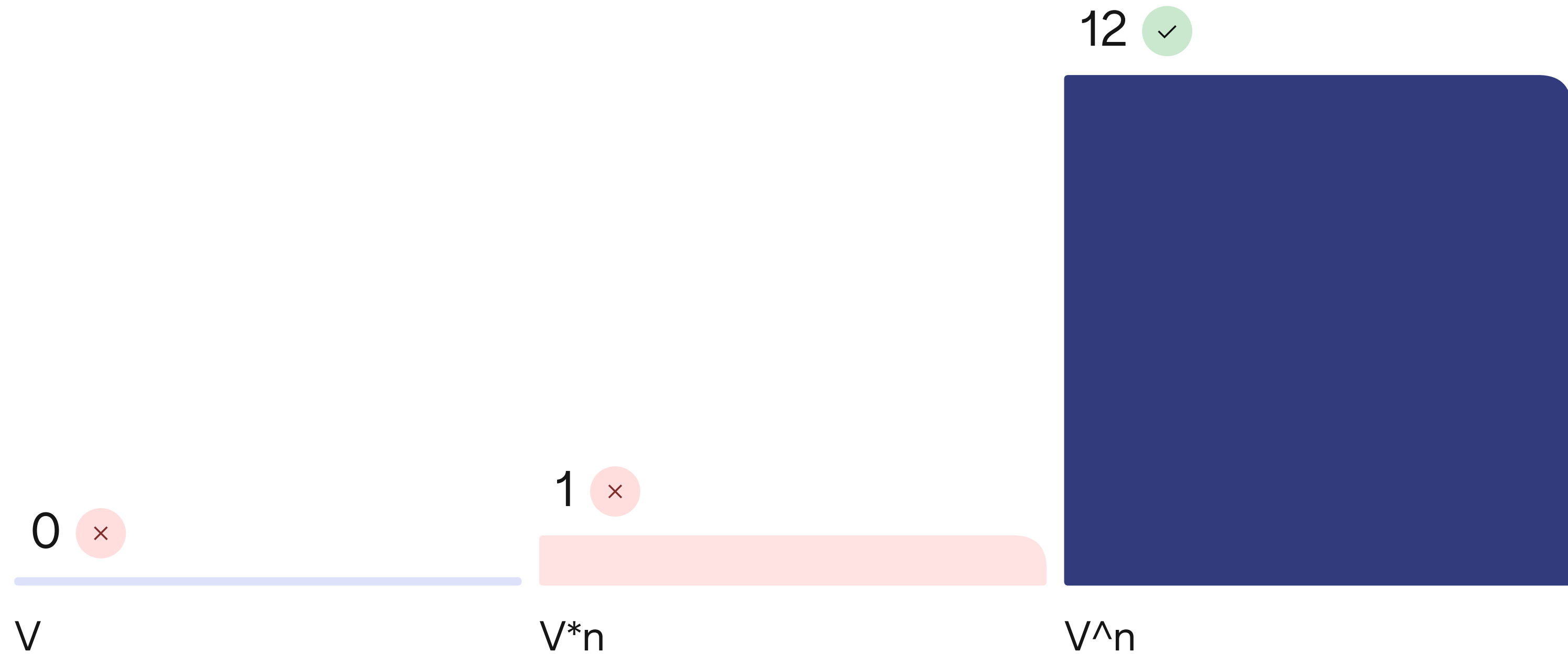
Unigram LM: size of tabular representation of $P(W|H)$



Bigram LM: size of tabular representation of $P(W|H)$



NGram LM: size of tabular representation of $P(W|H)$



Memorise phrases

The NGram LM makes up a sentence by gluing phrases, which it memorises in its tabular cpds along with their probabilities.

An increase in the order has an exponential cost: $V^n \rightarrow V^{n+1}$

Parameter estimation

Parameterisation: for each unique history $h \in \mathcal{W}^{n-1}$

$$W|H = h \sim \text{Categorical}(\pi_{1:V}^{(h)}) .$$

Parameter estimation given a dataset $\mathcal{D}$

$$\pi_w^{(h)} = \frac{\text{count}_{HW}(h, w)}{\sum_{o=1}^V \text{count}_{HW}(h, o)}$$

MLE for tabular Categorical cpds can be done by "count and divide"

Can you see the problem?

Data sparsity

The longer the history, the less likely it is that we have seen it.

Most of the history-word pairs will never be seen.

Tricks: smoothing, interpolation, backoff, etc.

Outline

- Language model
- NGram LM
 - Classic
 - Neural
- Evaluation

Neural parameterisation

The neural NGram LM uses a neural network $\mathbf{g}$ to parameterise cpds in context:

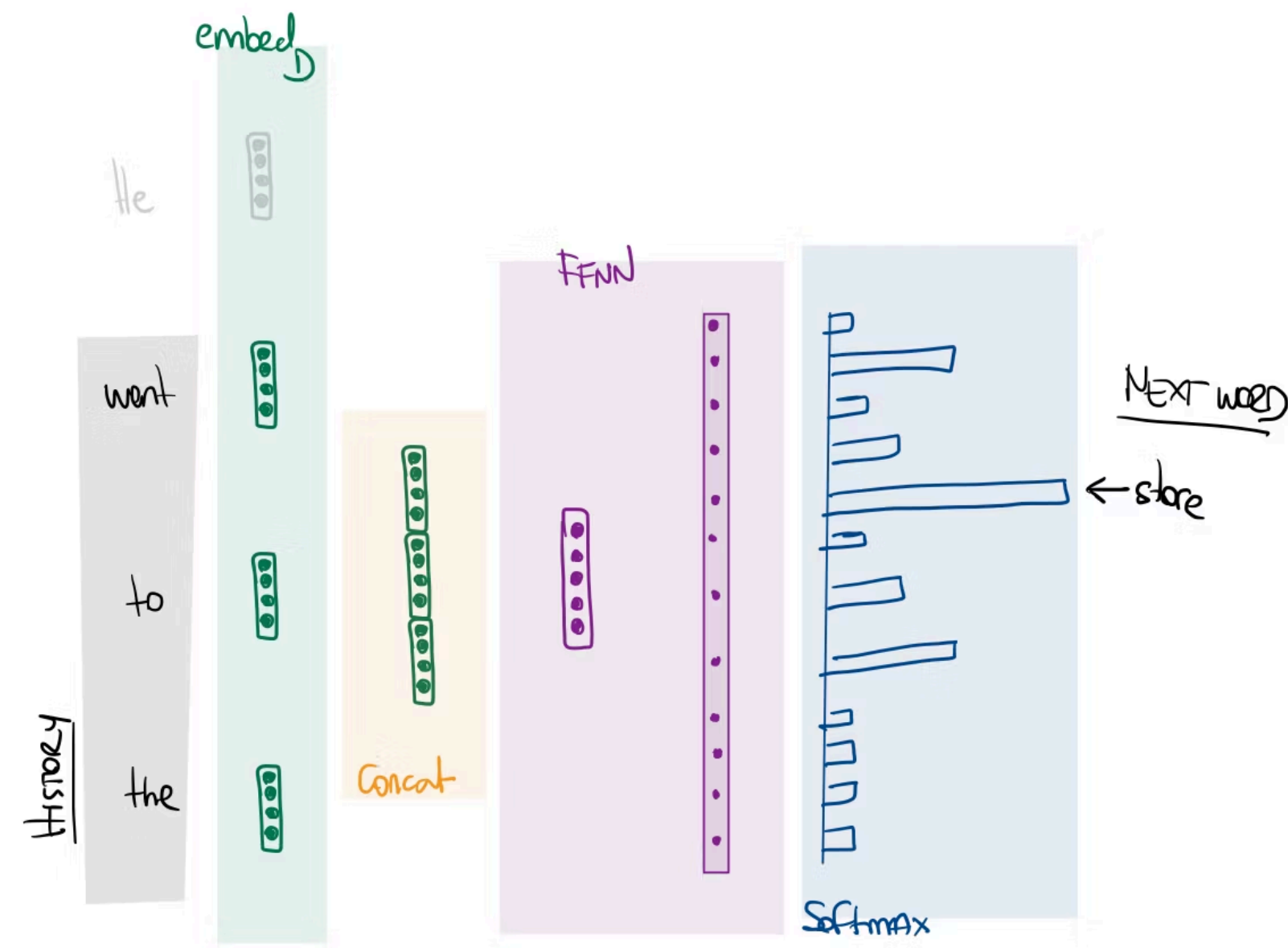
$$W|H = h \sim \text{Categorical}(\boldsymbol{\pi}(h; \boldsymbol{\theta}))$$

We store the parameters $\boldsymbol{\theta}$ of this neural network instead of the probabilities of the history-word pairs.

Next-Word Prediction

$$W|H = h \sim \text{Categorical}(\pi(h; \theta))$$

This is similar to a text classification problem: instead a document, we condition on history, instead of a distribution over K classes we predict a distribution over V words.



Architecture (4-gram LM)

- Use only the recent history
- Embed the words in it
- Concatenate their embeddings
- Use a FFNN to combine features and predict V logits
- Use softmax to parameterise a Categorical cpd

$$\begin{aligned} W|H = w_{i-n+1:i-1} &\sim \text{Categorical}(\boldsymbol{\pi}(w_{i-n+1:i-1}; \theta)) \\ \mathbf{e}_j &= \text{emb}_D(w_j; \theta_{\text{in}}) \\ \mathbf{u}_i &= \text{concat}(\mathbf{e}_{i-n+1}, \dots, \mathbf{e}_{i-1}) \\ \mathbf{s}_i &= \text{ffnn}_V(\mathbf{u}_i; \theta_{\text{out}}) \\ \boldsymbol{\pi}(w_{i-n+1:i-1}; \theta) &= \text{softmax}(\mathbf{s}_i) \end{aligned}$$

Architecture

A simple FFNN architecture that maps the concatenated embeddings of the $n - 1$ words that precede the next word to the conditional probabilities of each candidate word in the vocabulary.

Size of model (in number of parameters)

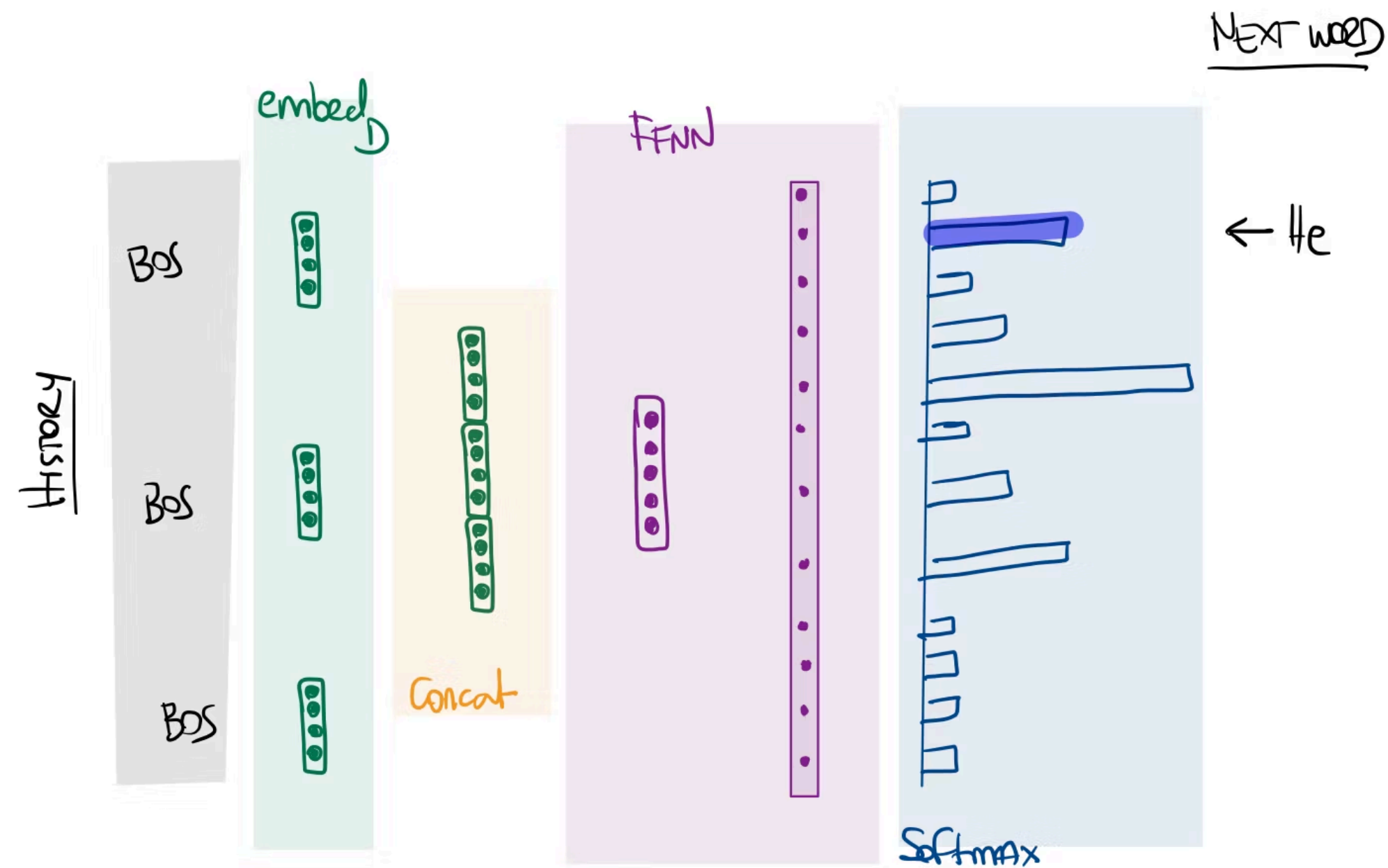
- Embedding layer: $V \times D$
- Concatenate layer: no parameters
- FFNN input-to-hidden: $(n - 1) \times D \times H$
- FFNN hidden-to-output: $H \times V$
- Softmax: no parameter

Parameter estimation

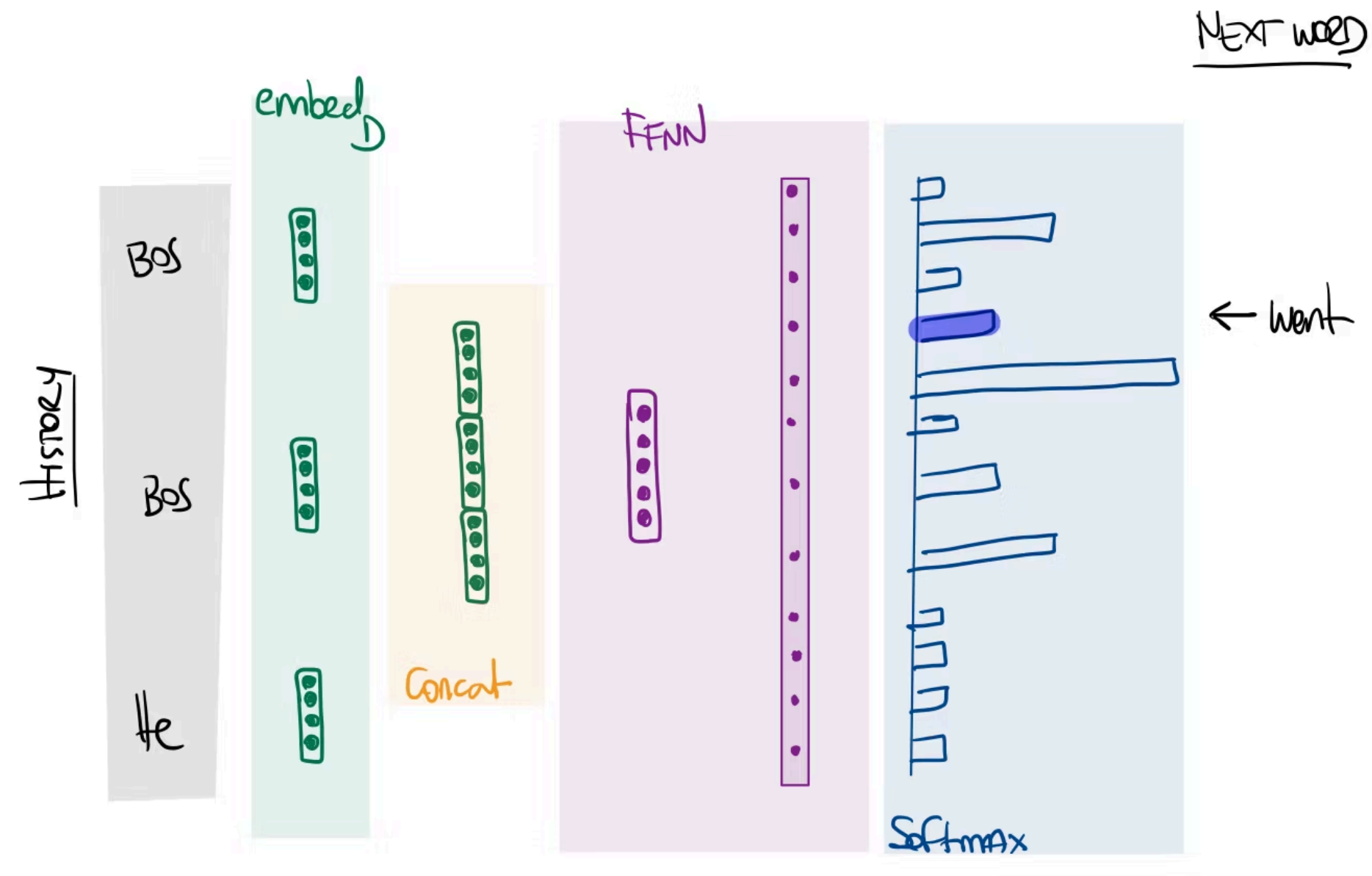
We optimise the log-likelihood of the NN parameters given a dataset of sentences.

For a single sentence

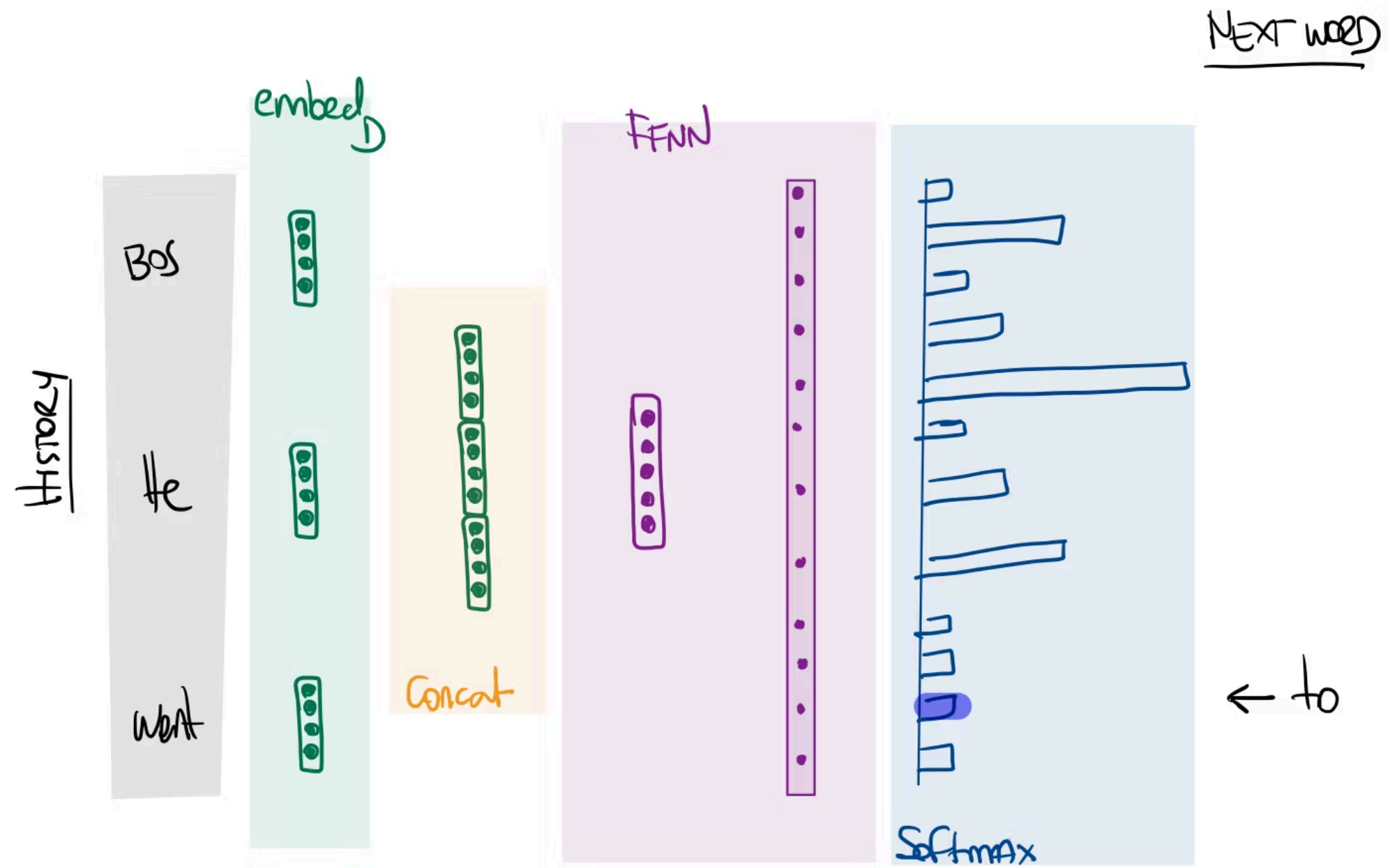
$$\begin{aligned}\mathcal{L}(\theta|w_{1:l}) \\ &= \sum_{i=1}^l \log \text{Cat}(w_i | \boldsymbol{\pi}(w_{i-n+1:i-1}; \theta)) \\ &= \sum_{i=1}^l \log [\boldsymbol{\pi}(w_{i-n+1:i-1}; \theta)]_{w_i}\end{aligned}$$



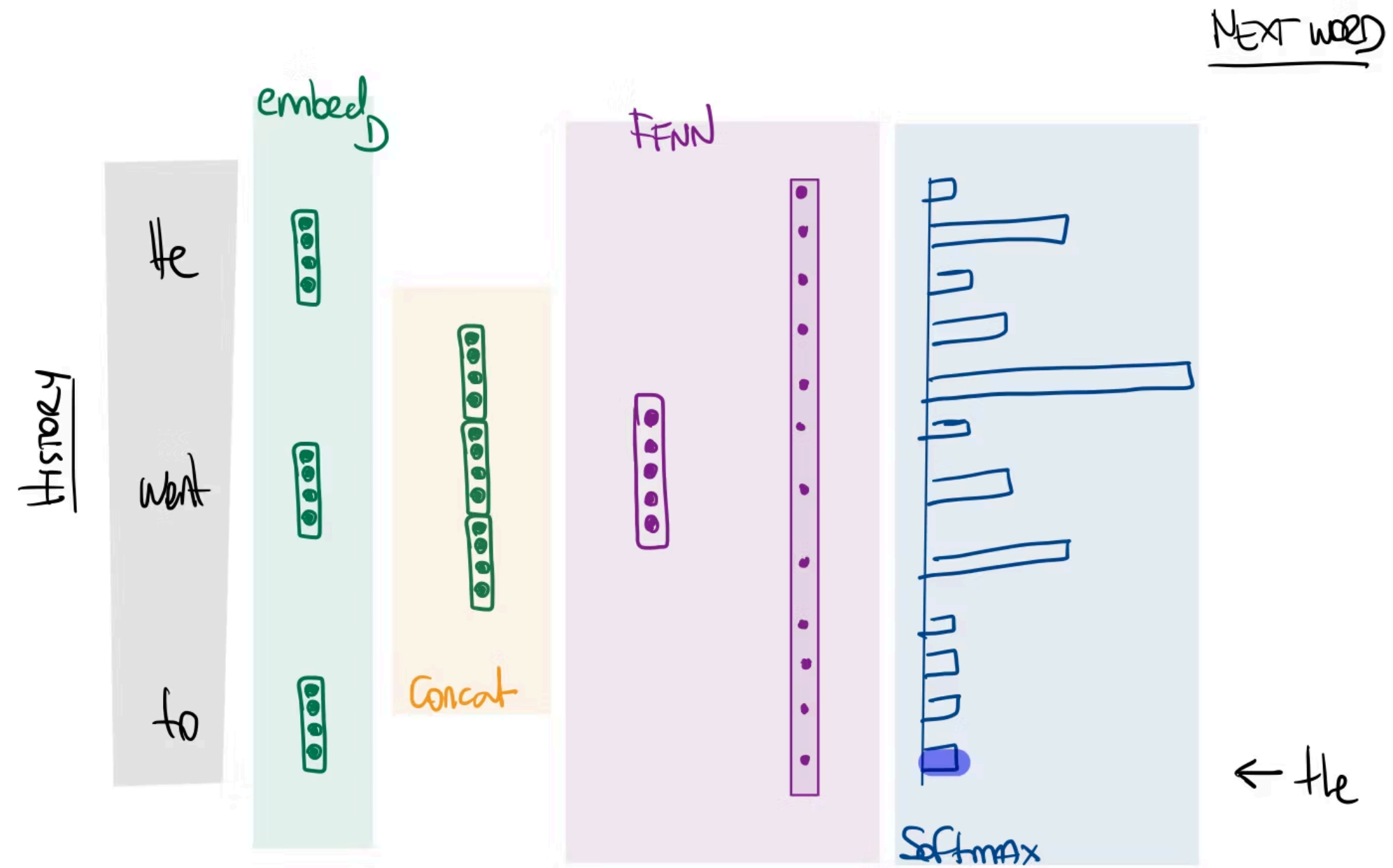
In the beginning, we have no context, but we have a fixed size history



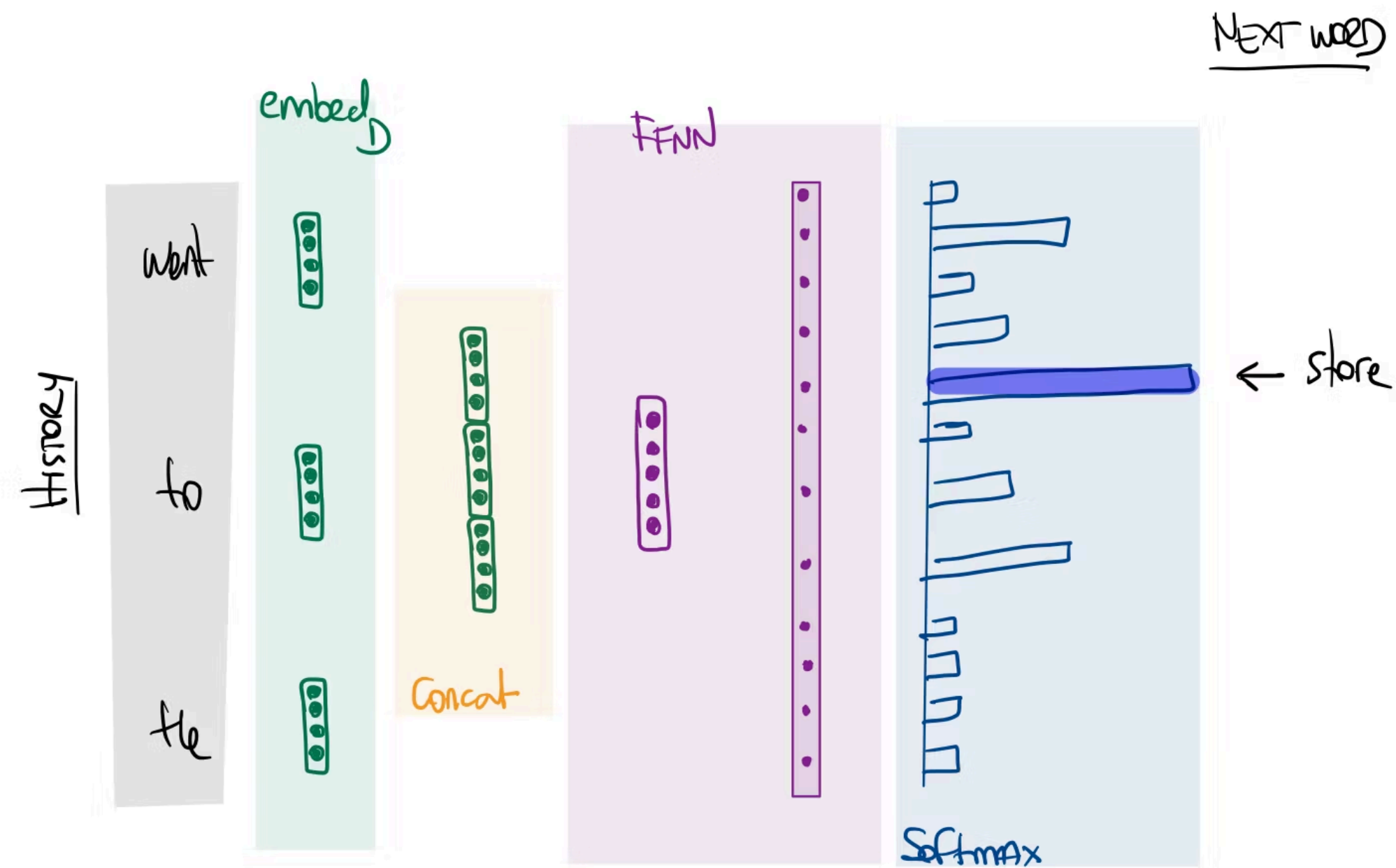
Then we get one word in the history



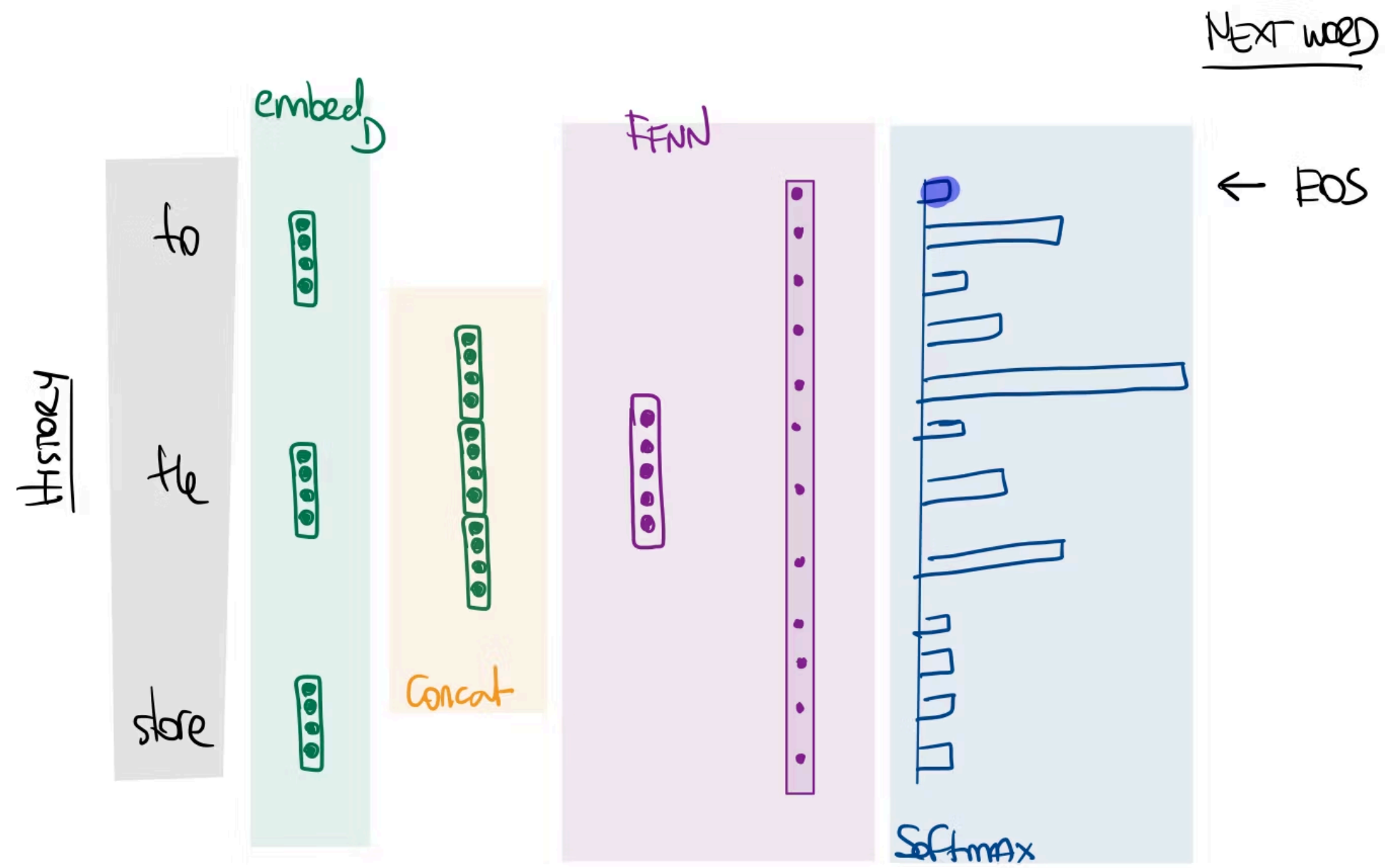
Then 2



Then 3



Now we start discarding older history



Until we generate the EOS symbol

Same factorisation, different parameterisation

Both models factorise the probability in the exact same way.

The key difference is the mechanism to parameterise the cpds.

NNs store embeddings and predict the encoding of a history and the probabilities of the candidates for next word.

Outline

- Language model
- NGram LM
 - Classic
 - Neural
- **Evaluation**

Intrinsic

We assess the model likelihood given
unseen sentences.

Often reported in the form of *perplexity*
(see T4).



Extrinsic

Plug the LM in a task (e.g., autocomplete)
and measure the performance of the task.

(see E4 and T4)



Statistical

Compare how the statistics of generated text distribute in relation to statistics of observed text.

(see T4)



Summary

- LMs are distributions over sentences
- Chain rule is the key to an LM
- Markov assumption (cond. indep.) leads to a viable model
- Classic approach uses inefficient tabular cpds
- Modern approach parameterises cpds using NNs

What next?

- T4: language models in Python
- Check the notebook before the class
- HC4b: applications, limitations and extensions
- WC4: exercises on applications and limitations